



Чему научиться в этом году?



minitower

7 фев в 01:39

Реализация MCP в Open WebUI. Часть 1. Интеграция с Open WebUI

Простой 11 мин 7.1K

Python*, Базы данных*, Визуализация данных*, Машинное обучение*

Кейс

Содержание

1. Введение
2. Поиски решения для web интерфейса
3. Интеграция с Open WebUI: первые шаги
4. Решение проблемы MCP с большими данными
5. Open WebUI Pipelines как расширение Open WebUI
 - 5.1. Возможности отображения в интерфейсе
 - 5.2. Переменные, получаемые в pipelines
 - 5.3. Вывод stream в Open WebUI
6. Заключение

Введение

В момент выхода протокола MCP нас очень заинтересовали его возможности. Нам хотелось использовать этот протокол для того, чтобы внутренние пользователи могли обращаться к базе данных в свободной форме и получать данные в течение нескольких минут. MCP для этого выглядел очень хорошо: пользователь может сформировать запрос на удобном для него языке, а LLM поймет, что нужно сделать и сделает это.

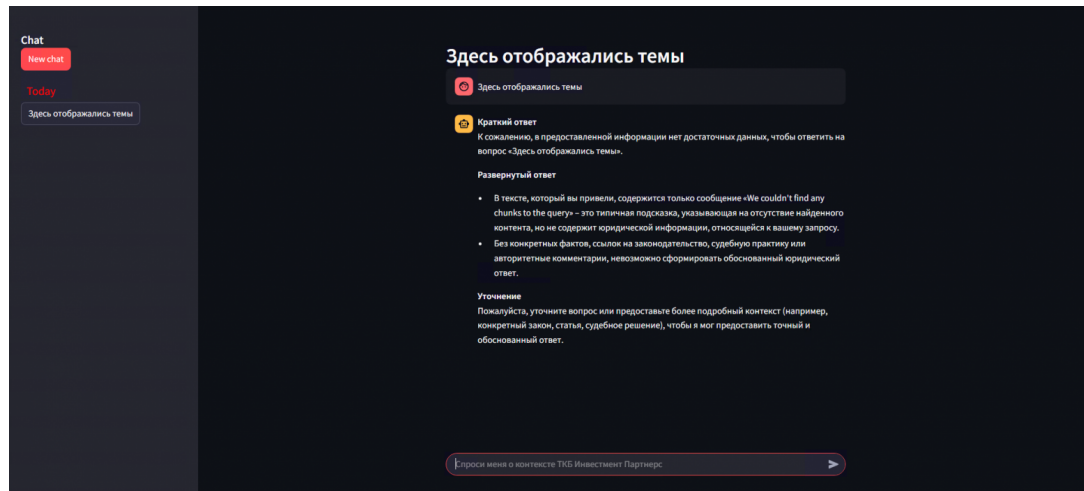
Особым вдохновением для нас стала [статья](#) о возможностях ClickHouse MCP (мы пользуемся именно этой базой данных). В рамках представленного демонстрационного материала было получение данных из ClickHouse с их дальнейшей визуализацией и отображением. Поэтому как только появилась возможность развернуть локальный инференс (ведь никто не хочет отдавать данные из своей БД на сторону, верно?), мы сразу приступили к его реализации. Для MVP было решено, что программа должна:

1. По запросу пользователя на естественном языке правильно отдавать данные;
2. По запросу пользователя данные должны быть визуализированы и представлены в виде графика;
3. Пользователь должен иметь возможность скачать их.

На первый взгляд MCP в связке с LLM полностью закрывал данные проблемы, однако с ростом объема данных стало заметно, что LLM не удастся обрабатывать их быстро и качественно, а написание SQL запросов для нее не всегда легкая задача (даже если примеры этих запросов у нее есть). В итоге мы получили потерю контроля над контекстом модели и непредсказуемый результат. В этой части я расскажу про интеграцию с Open WebUI и какая архитектура модели позволила победить вышеуказанные проблемы. Следующие статьи расскажут о реализации MCP таким, каким он позволяет выполнять наши задачи (но не финальной версии). Эта статья может быть полезна всем, кто строит свои модели на основе Open WebUI или еще только выбирает фреймворк, на котором предстоит строить будущую модель

Поиски решения для web интерфейса

К моменту написания нашего решения для протокола MCP у нас уже было решение для RAG системы, которая работала с некоторым контекстом внутри компании, а также были решения, которые были основаны на Streamlit (про это подробнее можно прочитать [здесь](#). Чат для RAG-а был написан полностью нами самостоятельно с помощью этого фреймворка и выглядел он примерно так:



RAG Streamlit интерфейс

Поэтому первой идеей было использовать тот же чат и для общения в формате MCP. Однако, для того, чтобы это было возможно, нужно было приложить еще некоторые усилия, а именно:

1. Написать алгоритм отображения графика (что для Streamlit не проблема);
2. Написать кнопки для скачивания файлов, копирование текстов сообщений, кнопки взаимодействия с заголовками;
3. Реализовать ролевую модель для пользователей;
4. Улучшенное взаимодействие с пользователем;
5. Реализацию stream отображения и прочее

В общем, работы по доработке интерфейса было достаточно много, а результат надо было показать еще вчера, поэтому от идеи писать его самому мы отказались (хотелось найти что-то готовое). Таким образом мы пошли на поиски фреймворка, который бы был "швейцарским ножом" для всего вышеперечисленного. Наткнулись на [Open WebUI](#) который позволял реализовать все, что нам хотелось (подключить локальный инференс, подключить любую модель, даже подключить MCP сразу из коробки). Исходя из выбора писать интерфейс самим (что потребовало бы много времени) и готовым инструментом было принято решение выбрать второе (хоть и пришлось потом подстраиваться под их правила).

Интеграция с Open WebUI: первые шаги

После того как мы развернули Open WebUI нам предстояло подключить к нему нашу модель. Благо Open WebUI поддерживает общение через OpenAI API, что позволило сделать это очень просто: мы развернули модель на нужной машине (с использованием llama-server) и подключили его прямо в интерфейсе Open WebUI:

The screenshot shows a dark-themed dialog box titled "Добавить соединение" (Add connection). It contains several input fields and a "Сохранить" (Save) button at the bottom right. The fields include: "Тип подключения" (Connection type) set to "Внешнее" (External); "URL" with a "Базовый адрес API" (Base API address) field; "Вход" (Auth) set to "Bearer" with a "Ключ API" (API key) field; "Headers" with a note "Enter additional headers in JSON format"; "ID префикса" (Prefix ID) with a note "Оставьте поле пустым, чтобы включить все модели из конечной точки '/models'" (Leave the field empty to include all models from the endpoint '/models'); "Тип поставщика" (Provider type) set to "Open AI"; and "IDs Модели" (Model IDs) with a "Добавить ID модели" (Add model ID) button. There is also a "Теги" (Tags) section with a "Добавить теги" (Add tags) button.

Подключение внешней модели

Сразу после реализации подключения модель отобразилась в списке моделей, где ее можно было редактировать:



Отображение модели

Сейчас у нас стоит gpt-oss, но первой нашей моделью была Qwen3. Проблема в ней была в том, что многие агентское поведение у данной модели было много хуже, чем в текущей, но об этом будет рассказано в следующих статьях

В рамках редактирования модели возможно выполнить следующее:

The screenshot shows a dark-themed interface for editing a model. The header includes "← назад" (Back) and "Доступ" (Access). The main content area is titled "gpt-oss-20b" and includes a description "Описание" (Description) and a "Добавить теги" (Add tags) button. Below this is a "Параметры модели" (Model parameters) section with a "Системный промпт" (System prompt) field and a "Расширенные параметры" (Advanced parameters) section. The advanced parameters section includes a "Выбор знаний" (Select knowledge) button, a "Загрузить файлы" (Load files) button, and a list of "Инструменты" (Tools) with checkboxes: "Clickhouse-Mcp-Test", "Clickhouse-Mcp", and "Atlassian-Mcp". There is also a "Возможности" (Capabilities) section with checkboxes for "Видение" (Vision), "Загрузка файла" (File upload), "File Context", "Веб-Поиск" (Web search), "Генерация Изображений" (Image generation), "Интерпретатор Кода" (Code interpreter), "Использование" (Usage), "Цитаты" (Quotes), "Обновления Статуса" (Status updates), and "Built-in Tools". At the bottom, there is a "Сохранить и обновить" (Save and update) button.

Редактирование моделей

1. Подключить различные инструменты (поиск, интерпретатор кода, генерация изображений или прочие кастомные инструменты, например MCP);
2. Подключить встроенный в Open WebUI RAG (он называется Знания, подробнее об этом режиме можно прочитать [тут](#));
3. Системный промпт;
4. Некоторые встроенные параметры модели (температуру, top_k, top_p и прочее).

Также модель может иметь некоторые встроенные функции, которые задаются прямо в Open WebUI (они позволяют расширять функционал модели некоторыми действиями, которые будут заложены в нее изначально, например, построить и отобразить график).

Сначала мы просто подключили модель к MCP и попытались задавать ей простые вопросы и это уже работало: модель ходила в MCP, получала данные и формировала ответ. Однако когда мы перешли от тестовых кейсов к реальным оказалось, что модель не очень хорошо понимает наши таблицы (даже с учетом комментариев к каждой строке) и не может посчитать даже среднюю доходность по портфелю.

Тогда мы решили воспользоваться функцией добавления Знаний, в которых записать примеры SQL запросов для модели. Промпт с примерами выглядел примерно следующим образом:

```
examples:
  [{
    prompt: "Найди доходность портфеля {portfolio_name} за период с {start_date} по {end_date}",
    query: WITH log_coef as (SELECT Portfolio, Dt,
      sum(log(TWR_dod+1)) OVER (
        PARTITION BY InvestmentPortfolioID
        ORDER BY Dt ASC ROWS BETWEEN UNBOUNDED PRECEDING AND 0 FOLLOWING
      ) AS TWR_cumulative_coef
    FROM Contribution.contribution_twr_1s_mcp
    WHERE lowerUTF8(Portfolio) LIKE '%{portfolio_name}%'
      AND Dt::date >={start_date}
      AND Dt::date <= {end_date}
    ORDER BY Dt DESC)
    SELECT Portfolio, Dt, exp(TWR_cumulative_coef) - 1 AS TWR_cumulative
    FROM log_coef;

    ,
    "tool_response": {
      "status": "success",
      "table": "contribution_twr_1s_port_portfolios",
      "columns": [
        {"name": "TWR_cumulative", "type": "Float64", "comment": "Доходность"}
      ],
      "count": 1,
      "rows": [
        {"TWR_cumulative": {query_result}}
      ]
    },
    "assistant_final": "Доходность для портфеля {portfolio_name} за период с {start_date} по {end_date} составляет {TWR_cumulative}%",
  }, ...]
```

[Объяснить с](#) 

В целом, это решило проблему, модель стала писать запросы лучше (не всегда идеально и недетерминированно, но это уже был результат). Следующей задачей было сформировать из этого график, который отобразится пользователю. Сначала мы добавили в промпт модели запрос использовать внутренний интерпретатор кода и написать код для графика и запустить его. Но

каково было наше огорчение, когда написанный код нужно было запускать вручную)
Увидев возможность реализации функций мы сразу решили, что это и будет нашей реализацией:

1. Модель получает запрос от пользователя;
2. Данные дополняются существующей моделью данных (которая содержит информацию о БД и запросах к ней);
3. Модель идет в MCP для получения данных и пишет SQL запрос;
4. Модель получает данные от MCP и вызывает необходимые функции;
5. Результат отображается пользователю.

К сожалению получилось так, что код самих функций не сохранился (т. к. они в определенный момент начали вызывать конфликты в Open WebUI) и мы их удалили. Поэтому примеров функций добавить не получится.

Эта связка отлично работала на небольших данных. Ежедневные данные за месяц (т. е. до 31 строки) обрабатывались очень хорошо и быстро, однако большие объемы данных (например, за год) начинали сильно отставать. Причина этому кроется в механизме вызова функции: модель писала запрос с помощью `<tool_calling>`, следовательно, скорость вызова функции напрямую зависела от того, сколько выходных токенов могла выдавать модель. В результате MCP отвечала максимально быстро, но вот вызов функции мог затягиваться на 30 и более секунд. К тому же это негативно сказывалось и на контексте для модели, т. к. какую-либо обработку этих данных делать было не нужно, однако она их получала.

Решение проблемы MCP с большими данными

Когда мы увидели это слабое звено мы начали искать похожие решения в интернете. Первое, на что мы обратили внимание было решение от Anthropic по типу [CodeAct](#). Оно предполагало, что модель сама пишет код, который затем исполняется интерпретатором Python, она получает результат, анализирует его и выдает пользователю. Тут решалась проблема, связанная с выводом в `<tool_calling>` большого массива информации (теперь информацию достаточно иметь в интерпретаторе), да и количество входных в модель данных также можно было порезать (чтоб контекст не забить).

Однако у этого подхода были и свои минусы:

1. **Сильно терялся контроль над действиями агента.** Если модель напишет код, который не соответствует нужным вычислениям, то тогда мы получим неверный ответ, а пользователям в данных вычислениях необходима точность;
2. **Модель может долго исправлять свои ошибки.** Усугублялось это тем, что используемые нами модели сравнительно небольшие (сейчас максимальный размер, который мы можем позволить - это 20B);
3. **Вывод пользователю недетерминирован.** Модель каждый раз пишет код вывода самостоятельно. В нашем алгоритме существует множество мест, где модель может "споткнуться", как следствие, неправильно предоставить результат.

Для того, чтобы обойти данные ограничения было принято решение сделать свой pipeline, который включал следующие шаги:

1. Модель имеет стандартный доступ к инструментам (вызывает их через `<tool_calling>`) и сообщает о том, что данные получены;
2. Скрипт собирает информацию из необходимого `<tool_calling>` и получает ответ инструмента;

3. Формируется нужный ответ для пользователя (график, если запрошено, таблица, ссылка на скачивание и прочее).

По сути это мало отличается от классического MCP, однако теперь модель не забивается лишним контекстом (т. к. мы его режем), а также мы можем визуализировать данные и делать с ними что захотим почти мгновенно после получения данных от MCP (остается небольшая задержка на route для графиков и email, но в целом результат дается в разы быстрее).

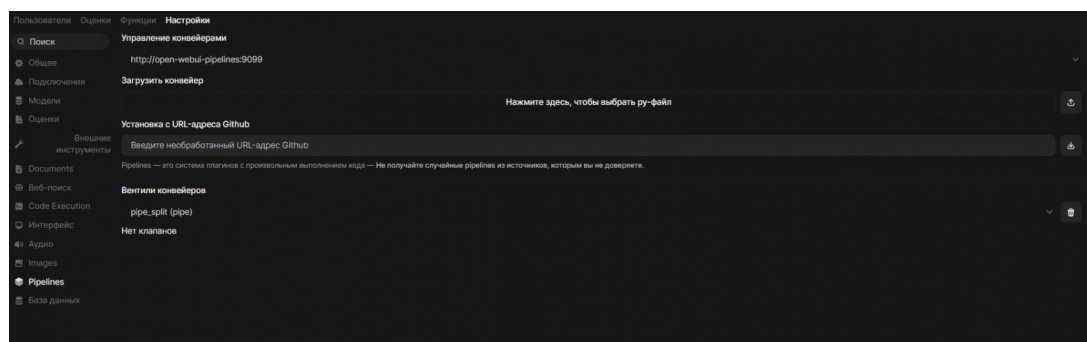
Однако стандартные средства Open WebUI не позволяли реализовать данный алгоритм. В поисках решения мы обратили внимания на расширение Open WebUI и обратили свой взор на Open WebUI Pipelines.

Open WebUI Pipelines как расширение Open WebUI

[Open WebUI Pipelines](#) - это отдельное приложение, которое может подключаться к основному Open WebUI и дополнять список моделей дополнительными моделями, написанные с помощью Python. Таким образом, это позволяет полностью контролировать логику общения с LLM и настраивать его так, как необходимо. Для того, чтобы освоение данного фреймворка пошло наиболее легко существуют [примеры](#), которые показывают, как можно использовать данный фреймворк. Схема выглядит примерно так:

1. Разворачивается Docker с данным образом;
2. Pipeline загружается в данный образ (либо через интерфейс Open WebUI, либо другими доступными способами);
3. Каждый раз при обращении к модели скрипт запускает функцию pipe, которая считает и выдает результат пользователю.

Наиболее простой способ загрузки - через сам Open WebUI:



Загрузка модели

Однако возможно загрузить его самому, прописав URL для pipelines внутри инициализации Docker образа (тогда они подгрузятся автоматически). В результате данных манипуляций появится новая модель, которая будет отражена в списке моделей:



Список моделей

Ее можно будет редактировать ровно так же, как саму модель, а все необходимые параметры будут переданы через переменную `body`, которая принимается функцией `pipelines`.

В данной интеграции стоит указать некоторые вещи, которые не указаны в примерах к данной библиотеке (на момент реализации этих фиц этого не было и в документации, для исследования

этих фич приходилось читать исходный код).

Возможности отображения в интерфейсе

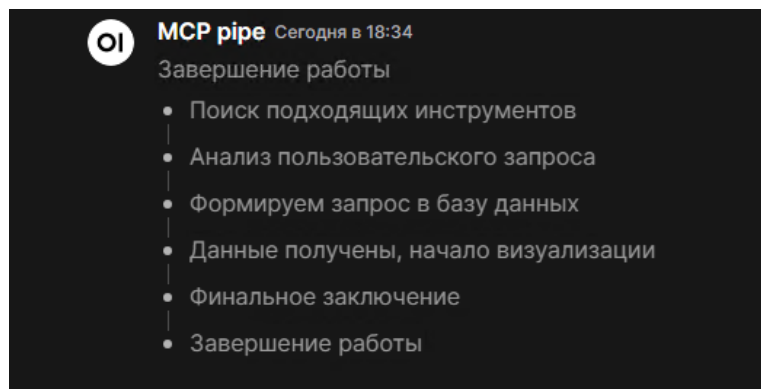
Автоматически Open WebUI не будет отрисовывать таблицы, HTML, base64 изображения и прочие прелести, которые в него были заложены при создании. Для того, чтобы это стало возможно необходимо использовать специальные ивенты.

Так, например, для того, чтобы было возможно отобразить статус определенного события (например, процесс получения данных), можно воспользоваться следующим ивентом:

```
## КОД def pipe() ##
yield {
  "event": {
    "type": "status",
    "data": {
      "description": "Поиск подходящих инструментов",
      "done": False,
    },
  },
}
## КОД def pipe() ##
```

Объяснить с 

Результатом будет следующая визуализация:



Статусы запроса

Если данный статус завершающий, то стоит отразить это в "done"=True .

Также для нашей задачи важен был ивент `embeds` , который показывает Open WebUI, что данный элемент: HTML страница, которую нужно отрисовать:

```
yield {
  "event": {
    "type": "embeds",
    "data": {
      "embeds": [f"""
        <!DOCTYPE html>
        <html>
        <head>
          <title>Hidden div</title>
          <style>
            {sql_style}
          </style>
        </head>
        <body>
```

```
        {sql_element.replace('{sql_query}', str(sql_query)),
          {sql_button_js}
        },
      ],
    ],
  ],
  "done": False,
},
},
}
```

Объяснить с 

В результате Open WebUI отрисовывает элемент в соответствии с тем, что передано в данном HTML. Так например, выглядит запрос доходности по определенному портфелю за 2025 год (выводится DataFrame, ссылка на скачивание, используемый запрос/инструмент и plotly график):



Пример вывода результата в Open WebUI

Также можно передавать более специфические ивенты, например, ~~запрос действий от пользователя~~ (меня корректно поправили в комментариях, что возможность запроса действий возможна только в функциях) или прикреплять файлы, однако в настоящий момент мы ими не пользовались.

Переменные, получаемые в pipelines

Open WebUI передает некоторые переменные во время запроса пользователя. В качестве данных переменных выступают:

1. **user_message** - строка, которая содержит последнее сообщение от пользователя;
2. **model_id** - id модели в Open WebUI;
3. **messages** - список сообщений в формате OpenAI API;
4. **body** - некоторая дополнительная информация относительно запроса (например, пользователь, который отправляет запрос).

Переменная body выглядит примерно следующим образом:

```
{ 'stream': False , --Нужен ответ stream или нет
  'model': 'pipe_stable_gpt_v3', -- Используемая модель
  'messages': [ -- Список сообщений в формате OpenAI API
    { 'role': 'user',
      'content': 'Сообщение' }
    ],
  'user': { --Информация о пользователе
    'name': 'имя',
    'id': '3012d212-ac22-4964-bcb8-a82acd7dd7b5',
    'email': 'email',
    'role': 'user'
  }
}
```

[Объяснить с](#) 

Данные переменные могут использоваться для целей персонализации, защиты данных в качестве ролей доступа и других целей.

У нас данная переменная используется для отправки файлов на почту (мы реализовали это не через отдельный tool, а функцией Python) и сохранении результатов в S3 для дальнейшего скачивания (если обратить внимание на результат модели, то можно обнаружить кнопку "Скачать CSV", эта функция сохраняет файл в S3 и создает публичную ссылку для скачивания файла).

Вывод stream в Open WebUI

Вывод ответа в качестве стрима в строку Open WebUI происходит по классической схеме ответа, при этом отправлять нужно не полный JSON, а только результат payload для отображения:

```
response = self.send_llm_req([
    { "role": "system", "content": final_prompt },
    { "role": "user", "content": user_message },
], stream=True)

for i in response.iter_lines():
    if len(i) > 0:
        chunk = json.loads(i[6:])
        try:
            yield chunk['choices'][0]['delta']['content']
        except KeyError:
            pass
        if chunk['choices'][0]['finish_reason'] == 'stop':
            break
```

[Объяснить с](#) 

Такой формат ответа позволяет выводить результат сразу в сообщение Open WebUI не дожидаясь его полной обработки.

Заключение

Open WebUI является очень хорошим фреймворком для работы с LLM-моделями, который позволяет развернуть графический интерфейс для взаимодействия с ними за несколько минут. При этом он не теряет в гибкости благодаря расширению Pipelines, которое позволяет полностью контролировать процесс разработки (в таком случае Open WebUI является только фронтендом для отображения и ничем более).

В ходе работы оказалось, что стандартная связка LLM -> MCP хорошо работает на небольших, демонстрационных данных, однако когда цель стоит в обработке больших массивов информации данная логика ломается, и использовать его становится невозможно. Да и LLM не всегда может написать тот запрос, который от нее хотят (даже если ей дали примеры этих самых запросов), а контекст забивается настолько быстро, что достаточно одного серьезного запроса для того, чтобы модель стала умирать.

При этом Pipelines оказались не просто расширением Open WebUI, а полноценным инструментом для построения кастомных LLM-сервисов с гибкой логикой, потоковым выводом, HTML-встраиваниями и доступом к пользовательскому контексту. Да, документация на момент реализации оставляла пробелы, и часть возможностей пришлось изучать через исходный код, однако сам подход показал себя устойчивым и хорошо масштабируемым.

В следующих частях данного цикла статей хотелось бы поподробнее остановиться на самом реализованном агенте: о том как он устроен у нас и как используется в настоящий момент в компании, а также о дальнейших шагах по его улучшению. В планах у нас реализовать агентское поведение (планировщик и еxecutor), реализовать деление на роли и безопасность использование агента. Надеюсь данная статья была вам полезна и вы прочитали ее с удовольствием (а, возможно, и открыли для себя что-то новое).

Только зарегистрированные пользователи могут участвовать в опросе. Войдите, пожалуйста.

Использовали ли вы ClickHouse MCP на «больших» данных (более 1000 строк)

0%	Да, использовал CodeAct	0
33.33%	Да, использовал похожее решение	1
0%	Да, использовал кастомное решение, напишу в комментариях	0
66.67%	Нет, но ваше решение мне нравится	2
0%	Нет, и ваше решение мне не нравится	0

Проголосовали 3 пользователя. Воздержались 2 пользователя.

Теги: open-webui, mcp, clickhouse-mcp, clickhouse, интеграции

Хабы: Python, Базы данных, Визуализация данных, Машинное обучение

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электронпочта

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



4K+

3

Охват за 30 дней

Карма

Влад Алферов @minitower

Аналитик данных и ML инженер

Подписаться



[GitHub](#)

Комментарии 3

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



runetfreedom

9 часов назад

Из-за критической уязвимости VLESS клиентов скоро все ваши VPN будут заблокированы

Простой

5 мин

135K

+319

269

410



zarazaexe

10 часов назад

BAREBONE2022: чтобы заблокировать этот протокол придется запретить MAX и Yandex

Простой

4 мин

40K

+94

147

62



Firemoon

8 часов назад

Вспоминаем молодость: как делались скриншоты на старых кнопочных Nokia

9 мин

6.5K

+33

6

2

**dalerank**

22 часа назад

Лишние вычисления

Простой

12 мин

12K

+31

46

18

**TrexSelectel**

4 часа назад

Беспроводная связь в ядерном пекле: как японцы заставили Wi-Fi выживать там, где умирает вся электроника

4 мин

4.7K

+28

6

0

**k0mar0v**

7 часов назад

Commodore OS Vision 3.0 — операционка для фанатов ретрогейминга

6 мин

4.8K

+22

7

0

**melnik909**

7 часов назад

9 CSS-лайфхаков для улучшения пользовательского опыта

Средний

6 мин

5K

Обзор

+20

14

1

**AmirUr**

7 часов назад

Управление SSH-доступом в 2026 году: от зоопарка с jump-host и Ansible к единой точке входа в инфраструктуру с Warpgate

Средний

6 мин

6.8K

Кейс

+17

33

4

**maxim_tsar**

9 часов назад

МАИ: как здесь учились конструкторы первых космических ракет

11 мин

4.4K

+16

5

0

**Sec-Explorer**

6 часов назад

«Олег, разверни тестовую базу»: как задача на 5 минут сорвала финтех-релиз и поссорила три отдела

6 мин

5.6K

Кейс

+15

12

7

От реле до нейросети: лучшие статьи сезона Heavy Digital

Турбо

Показать еще

МИНУТОЧКУ ВНИМАНИЯ



Турбо

Про IT на заводах: лучшие статьи сезона Heavy Digital



Турбо

Отношение айтишников к ИИ: результаты исследования



Турбо

УПС

Хочу – валю проект, хочу – помогаю: как чинить ИИ-агенты

ВОПРОСЫ И ОТВЕТЫ

Запрос на создание таблицы в clickhouse Yandex выдает ошибку 400 Bad Request, что не так с моим запросом?

PHP · Простой · 4 ответа

Почему Clickhouse читает много строк?

ClickHouse · Средний · 2 ответа

Возможно ли сделать систему логирования из ClickHouse, Logstash и Grafana?

Logstash · Средний · 3 ответа

Почему не могу подключиться к Clickhouse из интерфейса PHPStorm?

PhpStorm · Средний · 1 ответ

Ошибка с неизвестной таймзоной?

Go · Средний · 3 ответа

[Больше вопросов на Хабр Q&A](#)

ЧИТАЮТ СЕЙЧАС

Из-за критической уязвимости VLESS клиентов скоро все ваши VPN будут заблокированы

135K 410

Отчёт Cloudwards: Россия на предпоследнем месте по уровню свободы интернета в 2026 году

123K 202

Вышел клиент Telegram на Android v12.6.4 (6666), в андейте исправили проблему с работой MTProto

62K 59

BAREBONE2022: чтобы заблокировать этот протокол придется запретить MAX и Yandex

40K 62

Инцидент в сети «Ростелеком» с недоступностью сервисов произошёл из-за мощной DDoS-атаки

 44К  36

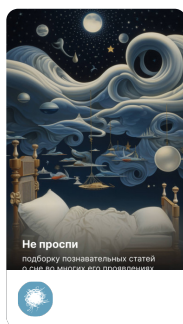
От реле до нейросети: лучшие статьи сезона Heavy Digital

Турбо

ИСТОРИИ



Лучшее из блогов компаний
за неделю



Самое время поспать



Просто взять и пошутить



Со статьями о Heavy Digital —
на завод



Пресс, который мы
заслужили



БЛИЖАЙШИЕ СОБЫТИЯ

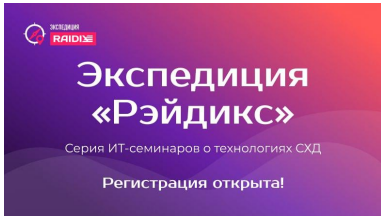


1 марта – 9 августа

**PROBA Awards —
международная
коммуникационная
премия, учрежденная в
2000 году**

Санкт-Петербург
[Маркетинг](#)

[Больше событий в календаре](#)



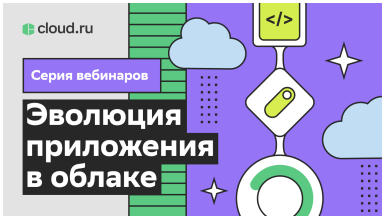
2 марта – 8 июня

Экспедиция «Рэйдикс»

Екатеринбург • Новосибирск •
Краснодар • Минск • Калининград •
Ростов-на-Дону

[Разработка](#) [Другое](#)

[Больше событий в календаре](#)

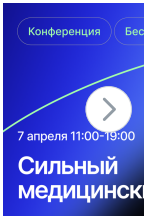


18 марта – 27 апреля

**Серия вебинаров
«Эволюция приложения
в облаке»**

Онлайн
[Разработка](#)

[Больше событий в календаре](#)



6 апреля

**Онлайн-к
«Сильный
бренд»**

Онлайн
[Менеджмент](#)

[Другое](#)

[Больше событий](#)

Ваш аккаунт

[Войти](#)
[Регистрация](#)

Разделы

[Статьи](#)
[Новости](#)
[Хабы](#)
[Компании](#)
[Авторы](#)
[Песочница](#)

Информация

[Устройство сайта](#)
[Для авторов](#)
[Для компаний](#)
[Документы](#)
[Соглашение](#)
[Конфиденциальность](#)

Услуги

[Корпоративный блог](#)
[Медийная реклама](#)
[Нативные проекты](#)
[Образовательные программы](#)
[Стартапам](#)



[Настройка языка](#)

[Техническая поддержка](#)

